

Practical – 3

Practical 3 AIM:

AIM: Implement Word Count / Frequency Program using MapReduce.

Problem Definition :

A digital publishing company stores thousands of news articles and blog posts in a Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations. As a Big Data Engineer, you are required to develop a MapReduce application that processes the text data and generates word frequency statistics.

Task 1:

Configure the Hadoop environment and upload the given text dataset to HDFS.

Task 2:

Develop and implement the Mapper program to extract words and generate intermediate key-value pairs in the form (word, 1).

Task 3:

Develop and implement the Reducer program to aggregate the occurrences of each word and generate the final word frequency count.

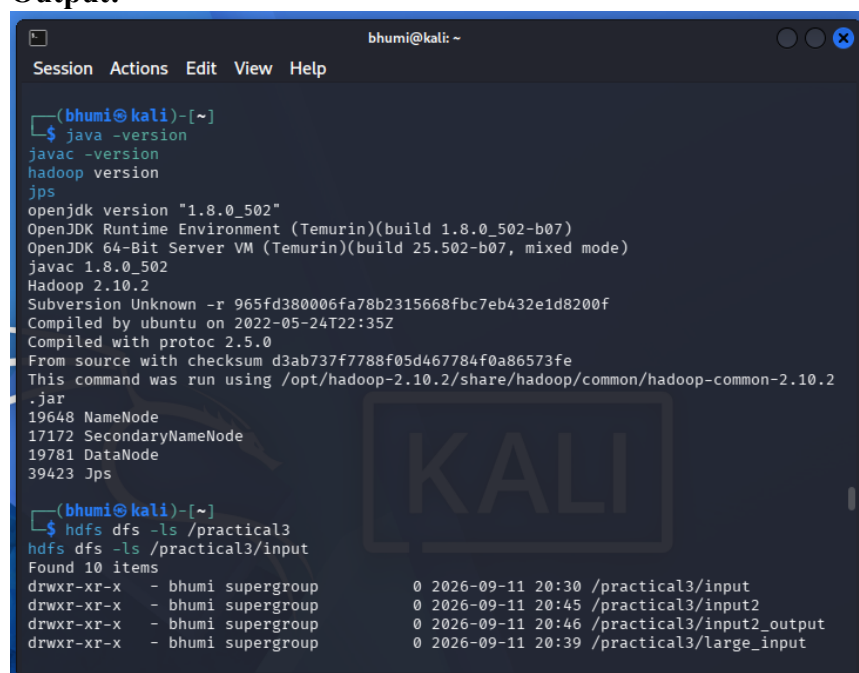
Task 4:

Execute the MapReduce job on Hadoop and store the output in HDFS. Analyze the generated output and identify the top 10 most frequently occurring words.

Task 5:

Evaluate the performance of the MapReduce application by executing it on datasets of different sizes and compare the processing time and output.

Output:



```
bhumi@kali: ~  
Session Actions Edit View Help  
[bhumi@kali]~  
$ java -version  
javac -version  
hadoop version  
jps  
openjdk version "1.8.0_502"  
OpenJDK Runtime Environment (Temurin)(build 1.8.0_502-b07)  
OpenJDK 64-Bit Server VM (Temurin)(build 25.502-b07, mixed mode)  
javac 1.8.0_502  
Hadoop 2.10.2  
Subversion Unknown -r 965fd380006fa78b2315668fbc7eb432e1d8200f  
Compiled by ubuntu on 2022-05-24T22:35Z  
Compiled with protoc 2.5.0  
From source with checksum d3ab737f7788f05d467784f0a86573fe  
This command was run using /opt/hadoop-2.10.2/share/hadoop/common/hadoop-common-2.10.2.jar  
19648 NameNode  
17172 SecondaryNameNode  
19781 DataNode  
39423 Jps  
[bhumi@kali]~  
$ hdfs dfs -ls /practical3  
hdfs dfs -ls /practical3/input  
Found 10 items  
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:30 /practical3/input  
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:45 /practical3/input2  
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:46 /practical3/input2_output  
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:39 /practical3/large_input
```

```

bhumi@kali: ~
Session Actions Edit View Help

drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:39 /practical3/large_output
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:48 /practical3/multi_input
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:49 /practical3/multi_output
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:35 /practical3/output
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:37 /practical3/output2
drwxr-xr-x - bhumi supergroup 0 2026-09-11 20:43 /practical3/stopword_output

Found 1 items
-rw-r--r-- 1 bhumi supergroup 164 2026-09-11 20:30 /practical3/input/input.txt

(bhumi@kali)-[~]
$ javac -classpath "${hadoop classpath}" Top10Words.java
ls -lh Top10Words*.class
-rw-rw-r-- 1 bhumi bhumi 1.8K Sep 11 21:00 'Top10Words$IntSumReducer.class'
-rw-rw-r-- 1 bhumi bhumi 1.8K Sep 11 21:00 'Top10Words$TokenizerMapper.class'
-rw-rw-r-- 1 bhumi bhumi 1.5K Sep 11 21:00 Top10Words.class

(bhumi@kali)-[~]
$ jar -cvf top10words.jar Top10Words*.class
ls -lh top10words.jar
added manifest
adding: Top10Words$IntSumReducer.class(in = 1742)(out= 742)(deflated 57%)
adding: Top10Words$TokenizerMapper.class(in = 1788)(out= 781)(deflated 56%)
adding: Top10Words.class(in = 1495)(out= 816)(deflated 45%)
-rw-rw-r-- 1 bhumi bhumi 3.1K Sep 11 21:00 top10words.jar

(bhumi@kali)-[~]
$ hdfs dfs -rm -r -f /practical3/output
time hadoop jar top10words.jar Top10Words /practical3/input /practical3/output

```

```

bhumi@kali: ~
Session Actions Edit View Help

(bhumi@kali)-[~]
$ hdfs dfs -cat /practical3/output/part-r-00000
a 1
analytics 1
and 1
big 4
data 5
fast 1
framework 1
hadoop 5
is 3
processes 1
processing 1
scalable 1
useful 1
using 1
with 1

(bhumi@kali)-[~]
$ time hadoop jar top10words.jar Top10Words /practical3/input /practical3/output2
26/09/11 21:01:54 INFO Configuration.deprecation: session.id is deprecated. Instead, use dfs.metrics.session-id
26/09/11 21:01:54 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTracker, sessionId=
Exception in thread "main" org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory hdfs://localhost:9000/practical3/output2 already exists
    at org.apache.hadoop.mapreduce.lib.output.FileOutputFormat.checkOutputSpecs(FileOutputFormat.java:146)
    at org.apache.hadoop.mapreduce.JobSubmitter.checkSpecs(JobSubmitter.java:279)

```

```

bhumi@kali: ~
Session Actions Edit View Help

(bhumi@kali)-[~]
$ time hadoop jar top10words.jar Top10Words /practical3/large_input /practical3/large_output
26/09/11 21:02:08 INFO Configuration.deprecation: session.id is deprecated. Instead, use dfs.metrics.session-id
26/09/11 21:02:08 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTracker, sessionId=
Exception in thread "main" org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory hdfs://localhost:9000/practical3/large_output already exists
    at org.apache.hadoop.mapreduce.lib.output.FileOutputFormat.checkOutputSpecs(FileOutputFormat.java:146)
    at org.apache.hadoop.mapreduce.JobSubmitter.checkSpecs(JobSubmitter.java:279)
    at org.apache.hadoop.mapreduce.JobSubmitter.submitJobInternal(JobSubmitter.java:145)
    at org.apache.hadoop.mapreduce.Job$11.run(Job.java:1573)
    at org.apache.hadoop.mapreduce.Job$11.run(Job.java:1570)
    at java.security.AccessController.doPrivileged(Native Method)
    at javax.security.auth.Subject.doAs(Subject.java:422)
    at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1938)
    at org.apache.hadoop.mapreduce.Job.submit(Job.java:1570)
    at org.apache.hadoop.mapreduce.Job.waitForCompletion(Job.java:1591)
    at Top10Words.main(Top10Words.java:74)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)

```

```

bhumi@kali: ~
Session Actions Edit View Help View image

real    10.19s
user    3.87s
sys     3.44s
cpu     71%

(bhumi@kali)-[~]
$ hdfs dfs -cat /practical3/large_output/part-r-00000
a      10
analytics      10
and      10
big      40
data      50
fast      10
framework      10
hadoop      50
is      30
processes      10
processing      10
scalable      10
useful      10
using      10
with      10

(bhumi@kali)-[~]
$ javac -classpath "${hadoop classpath}" Top10StopWords.java
jar -cvf top10stopwords.jar Top10StopWords*.class
hdfs dfs -rm -r -f /practical3/stopword_output
hadoop jar top10stopwords.jar Top10StopWords /practical3/input /practical3/stopword_output
hdfs dfs -cat /practical3/stopword_output/part-r-00000

```

Key Questions / Analysis

Q1. What is the role of Mapper and Reducer in the Word Count application?

- **Mapper:** Reads the input text and breaks it into words. It produces a key-value pair like (word, 1) for every word.

- **Reducer:** Receives the same words together and adds their values to find the total count of each word.
- Example: hello hello world → hello: 2, world: 1.

Q2. How are intermediate key-value pairs generated and processed?

- The Mapper reads the input data line by line.
- It separates the text into individual words.
- For each word, it generates a key-value pair such as (hello, 1).
- MapReduce groups all identical keys together.
- The Reducer then adds their values and produces the final word count.

Q3. How does HDFS store and distribute the input dataset?

- HDFS stores large files by dividing them into smaller **blocks**.
- These blocks are stored on different machines called **DataNodes**.
- HDFS also keeps copies of blocks for safety.
- This allows data to be stored and processed across multiple machines.

Q4. What advantages does MapReduce offer for processing large datasets?

- It can process very large amounts of data.
- It divides the work among multiple machines.
- It provides **parallel processing**, which makes processing faster.
- It can handle machine failures because HDFS stores multiple copies of data.
- It is suitable for distributed and large-scale data processing.

Q5. How does the execution time vary with dataset size?

- As the dataset size increases, the execution time generally also increases.
- However, MapReduce can divide the work among multiple machines, so the increase in time is less compared to processing everything on a single machine.
- With more machines and parallel processing, large datasets can be processed efficiently.

Supplementary Problems

Find the top 10 most frequently occurring words. Exclude stop words (e.g., “a,” “an,” “the,” “is”) from the analysis. Compare word frequency results for multiple input files.

Output:

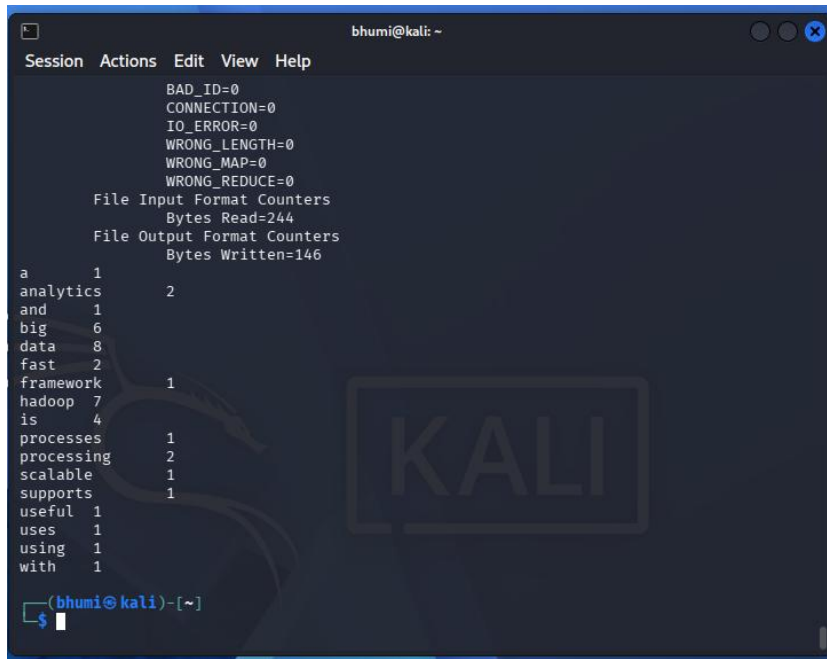
```

bhumi@kali: ~
Session Actions Edit View Help
useful 1
using 1
with 1

(bhumi@kali)-[~]
$ hdfs dfs -ls /practical3/multi_input
Found 2 items
-rw-r--r-- 1 bhumi supergroup 164 2026-09-11 20:48 /practical3/multi_input/in
put.txt
-rw-r--r-- 1 bhumi supergroup 80 2026-09-11 20:48 /practical3/multi_input/in
put2.txt

(bhumi@kali)-[~]
$ hdfs dfs -rm -r -f /practical3/multi_output
hadoop jar top10words.jar Top10Words /practical3/multi_input /practical3/multi_output
hdfs dfs -cat /practical3/multi_output/part-r-00000
Deleted /practical3/multi_output
26/09/11 21:03:46 INFO Configuration.deprecation: session.id is deprecated. Instead, u
se dfs.metrics.session-id
26/09/11 21:03:46 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTr
acker, sessionId=
26/09/11 21:03:47 WARN mapreduce.JobResourceUploader: Hadoop command-line option pars
ing not performed. Implement the Tool interface and execute your application with ToolR
unner to remedy this.
26/09/11 21:03:47 INFO input.FileInputFormat: Total input files to process : 2
26/09/11 21:03:47 INFO mapreduce.JobSubmitter: number of splits:2
26/09/11 21:03:47 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local646
082293_0001
26/09/11 21:03:48 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
26/09/11 21:03:48 INFO mapreduce.Job: Running job: job_local646082293_0001

```



```
bhumii@kali: ~  
Session  Actions  Edit  View  Help  
BAD_ID=0  
CONNECTION=0  
IO_ERROR=0  
WRONG_LENGTH=0  
WRONG_MAP=0  
WRONG_REDUCE=0  
File Input Format Counters  
  Bytes Read=244  
File Output Format Counters  
  Bytes Written=146  
a          1  
analytics  2  
and        1  
big        6  
data       8  
fast       2  
framework  1  
hadoop     7  
is         4  
processes  1  
processing 2  
scalable   1  
supports   1  
useful     1  
uses       1  
using      1  
with       1  
  
(bhumii@kali)-[~]  
$
```

Key Skills to be Addressed

Hadoop Ecosystem Fundamentals | HDFS Operations | MapReduce Programming | Distributed Data Processing | Data Analysis and Interpretation

Tools /Technology

Hadoop 3.x | HDFS | Java JDK 8/11 | Eclipse/IntelliJ IDEA | Linux/Ubuntu Environment